



HAWK

SECURITY IDE / LOCAL OPERATOR WORKSPACE

GUIDE COMPLET DU PROJET

Comprendre Hawk. Maitriser chaque surface. Prouver chaque changement.

Architecture, interface, Hawk AI, scanners, MCP, Docker,
Browser/Burp, options, API, sécurité, release et workflows.

VERSION

0.7.0

COMMIT

ddf4c51

SURFACE

LOCAL-FIRST

DATE

20 JUILLET 2026

Document généré depuis le dépôt local Hawk et ses interfaces rendues.

Usage prévu: machine personnelle, projets autorisés, contrôle humain.

UNDERSTAND CODE -> OBSERVE TRAFFIC -> PROVE -> FIX -> RETEST

github.com/MrBoodj011/Hawk-IDE



Identité du document

Base analysée

Dépôt local Hawk version 0.7.0, commit ddf4c51. Le contenu est fondé sur le code, les manifestes, la documentation et les interfaces réellement rendues. Les captures ne sont pas des maquettes marketing.

243 fichiers TypeScript/TSX	95 fichiers de tests	24 commandes Hawk dans l'IDE	24 réglages configurables
49 boutons et contrôles UI statiques	3 modèles de scan gouverné	5 règles d'audit passif	32 workers Docker simultanés max

Comment lire ce guide

- Parties 1 à 4: comprendre le produit, l'architecture et l'installation.
- Parties 5 à 10: utiliser Mission Control, Hawk AI, le moteur de code, la sécurité, MCP et Docker.
- Annexes: retrouver chaque bouton, commande, réglage, endpoint, outil MCP, option CLI et lien.
- État réel: les limites et actions externes restantes sont explicitement séparées des fonctions déjà implémentées.

Dépôt: github.com/MrBoodj011/Hawk-IDE - Liaison health.json: [Cybrense Hawk](#)



NAVIGATION

Sommaire

Identité du document	2
Comment lire ce guide	2
Sommaire	3
1. Hawk en une phrase et en détail	6
Ce que Hawk fait	6
Ce que Hawk ne promet pas	6
2. Surfaces du produit	7
3. Architecture et circulation des données	8
Séquence normale	8
Composants du dépôt	9
4. Installation, lancement et premier démarrage	10
Pré-requis de développement	10
Construction depuis le dépôt	10
Lancer le daemon seul	10
Préparer le worker Docker local	10
Premier démarrage Local AI	10
5. Mission Control - lecture de l'interface	11
Les six indicateurs	11
Navigation latérale	12
6. Tous les boutons de Mission Control	13
7. Hawk AI - le workspace d'ingénierie	14
Contexte sélectionnable	14
Cycle d'une session	14
8. Tous les contrôles de Hawk AI	16
Événements visibles dans le flux	16
9. Hawk Coding Core	17
Contrat mémoire	17
Debugger Agent	17
10. Modèles LLM et Local AI	18

Politique des clés	18
Installation Ollama contrôlée	18
11. Workflow sécurité de bout en bout	19
Indexation et règles passives	19
Templates de scan	19
12. Findings, reproduction et retest	20
Cycle de vie d'un finding	20
Sandbox de reproduction	20
Neuf gates de vérification	21
13. Browser et Burp Companions	22
Browser Companion	23
Burp Companion	23
Identity Replay gouverné	23
14. Evidence Builder et Security Graph	24
Noeuds du graphe	24
Exemples de navigation	24
15. Smart MCP Brain	25
Contrat de goal	25
Outils Smart MCP	25
16. MCP - ressources, prompts et outils techniques	26
Ressources	26
Prompts	26
Outils MCP IDE et Docker	27
Outils MCP Browser	27
17. Docker Worker Mesh et récupération	28
Capacité et scheduling	28
Isolation de chaque worker	28
Crash, restart et network failure	28
18. Réglages Hawk - catalogue complet	29
19. Command Palette et raccourcis	30
Raccourcis	30
20. API locale du daemon	31
21. CLI Hawk	33

Commandes slash du TUI	33
Outils du CLI	33
22. Données locales, sécurité et confidentialité	34
Hardening implémenté	34
Politiques	34
23. Build desktop, release et auto-update	35
Update réel	35
Actions owner restantes	35
24. Workflows pratiques	36
A. Corriger une fonctionnalité avec Hawk AI	36
B. Examiner un signal de sécurité	36
C. Distribuer une tâche longue	36
D. Utiliser Hawk pour bug bounty autorisé	36
25. Validation actuelle, limites et roadmap	37
Limites délibérées ou restantes	37
Roadmap technique	37
26. Liens et références	38
Glossaire rapide	38
27. Conclusion et point de départ	39
Par où commencer	39



01 / VISION

1. Hawk en une phrase et en détail

Hawk est un IDE de développement et de sécurité basé sur Code-OSS. Il rassemble l'assistance AI de type Cursor, l'analyse passive du code, la capture Browser/Burp, un graphe de preuve, des workflows MCP gouvernés, des workers Docker isolés et des rapports portables.

En clair

Ce n'est pas simplement VS Code avec un scanner. Hawk essaie de conserver une chaîne complète: comprendre le code -> observer les requêtes -> identifier un signal -> collecter une preuve -> corriger -> tester -> retester.

Ce que Hawk fait

- Édite du code avec un agent AI dans un worktree Git isolé.
- Affiche le plan, les outils, la réponse en streaming, l'historique, le diff exact et les tests.
- Prédit des complétions et des modifications multilignes avec Hawk Tab / Next Edit.
- Indexe localement les symboles, types, imports, appels et routes.
- Détecte des signaux AppSec passifs sans exécuter le projet.
- Corrèle le code, les routes, le trafic observé, les findings, les preuves, les patches et les tests.
- Reproduit certains signaux déterministes dans un sandbox Docker sans réseau.
- Planifie des missions MCP typées avec portée, budget, approbation et hash immuable.
- Distribue des tâches indépendantes entre plusieurs workers Docker avec reprise après incident.
- Génère des rapports Markdown, HTML, JSON, SARIF et un manifest SHA-256.

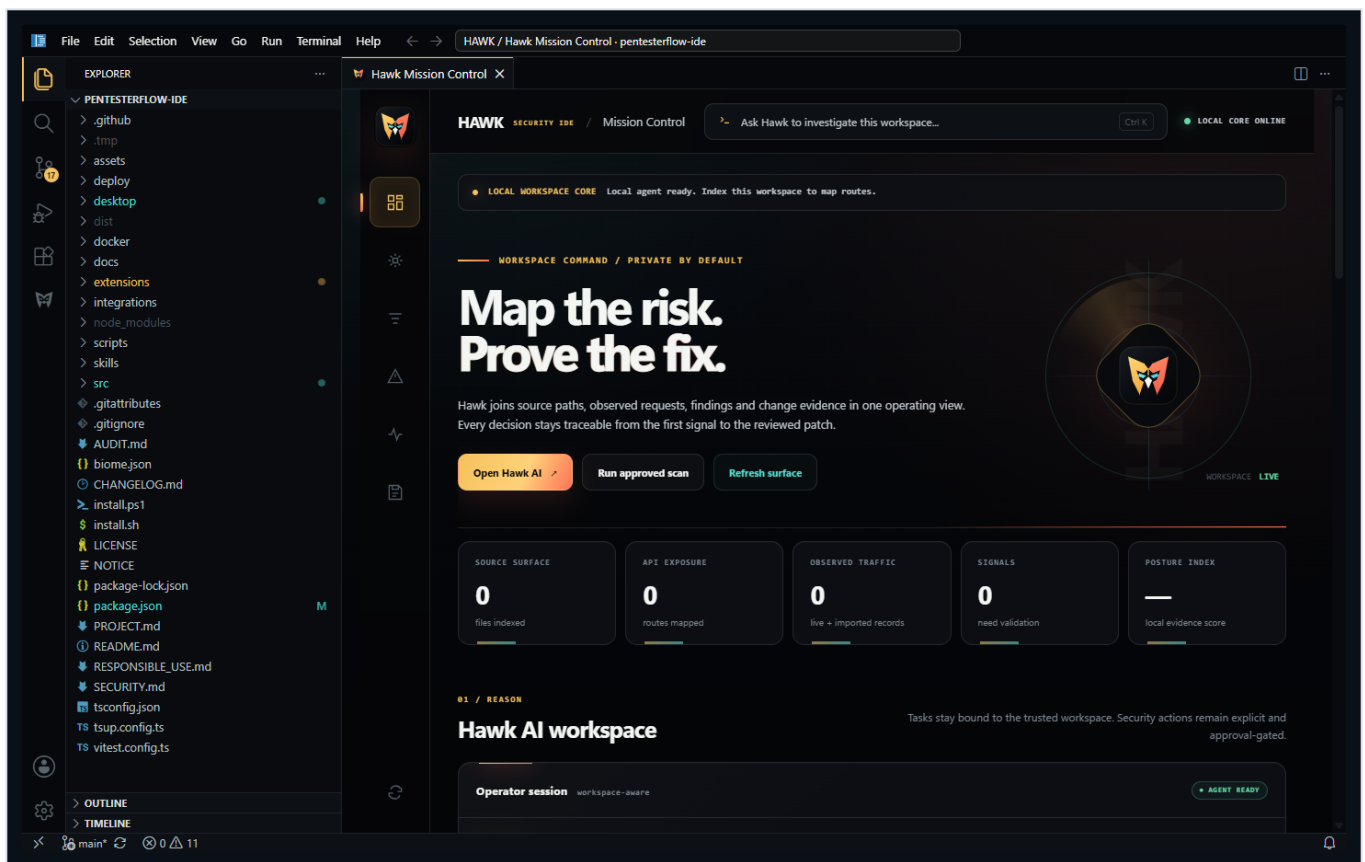
Ce que Hawk ne promet pas

- Il ne garantit pas de trouver toutes les vulnérabilités ou tous les bug bounties.
- Un signal statique n'est pas automatiquement une vulnérabilité confirmée.
- Une reproduction sandbox n'est pas automatiquement une preuve d'impact réel.
- Hawk ne donne pas d'autorisation de tester une cible: la portée doit déjà être autorisée.
- Il n'existe pas de cloud Hawk, compte, équipe, Stripe, licence, synchronisation ou télémétrie.
- Le pentest externe indépendant reste une action humaine séparée.

02 / PRODUIT

2. Surfaces du produit

Surface	Rôle	Frontière
Hawk Desktop	Application Code-OSS brandée, éditeur, terminal, Git, debug et extension Hawk intégrée.	Confiance du workspace + token local.
Mission Control	Vue opérationnelle: routes, trafic, signaux, preuves, posture, MCP et actions.	Les signaux restent suspects jusqu'à validation.
Hawk AI	Agent de code en streaming, contexte éditeur, historique, diff, tests, Apply/Reject/Revert.	Modifications isolées et revue manuelle.
Hawk Coding Core	Hawk Tab, Next Edit, index AST/type, recherche, benchmark, debugger agent, merge.	Code local, modèle local ou BYOK explicite.
Local AI	Découverte/installation Ollama, choix de modèle selon la RAM, configuration loopback.	Digest, taille et signature Windows vérifiés.
Browser Companion	Capture Fetch/XHR/WebSocket et webRequest sous portée explicite.	Désactivé par défaut, token, regex, rate limit.
Burp Companion	Transmet le trafic Burp explicitement autorisé vers Hawk.	Portée Burp, redaction, file bornée.
Smart MCP Brain	Goals, policy, DAG, modèles, approbations, runs, preuves et mémoire.	Autorité, budget et hash immuables.
Docker Worker Mesh	Exécution parallèle de tâches indépendantes et récupération.	Image locale, non-root, no-network par défaut.
Evidence Builder	Rapports et manifestes portables.	Métadonnées assainies, aucun replay.

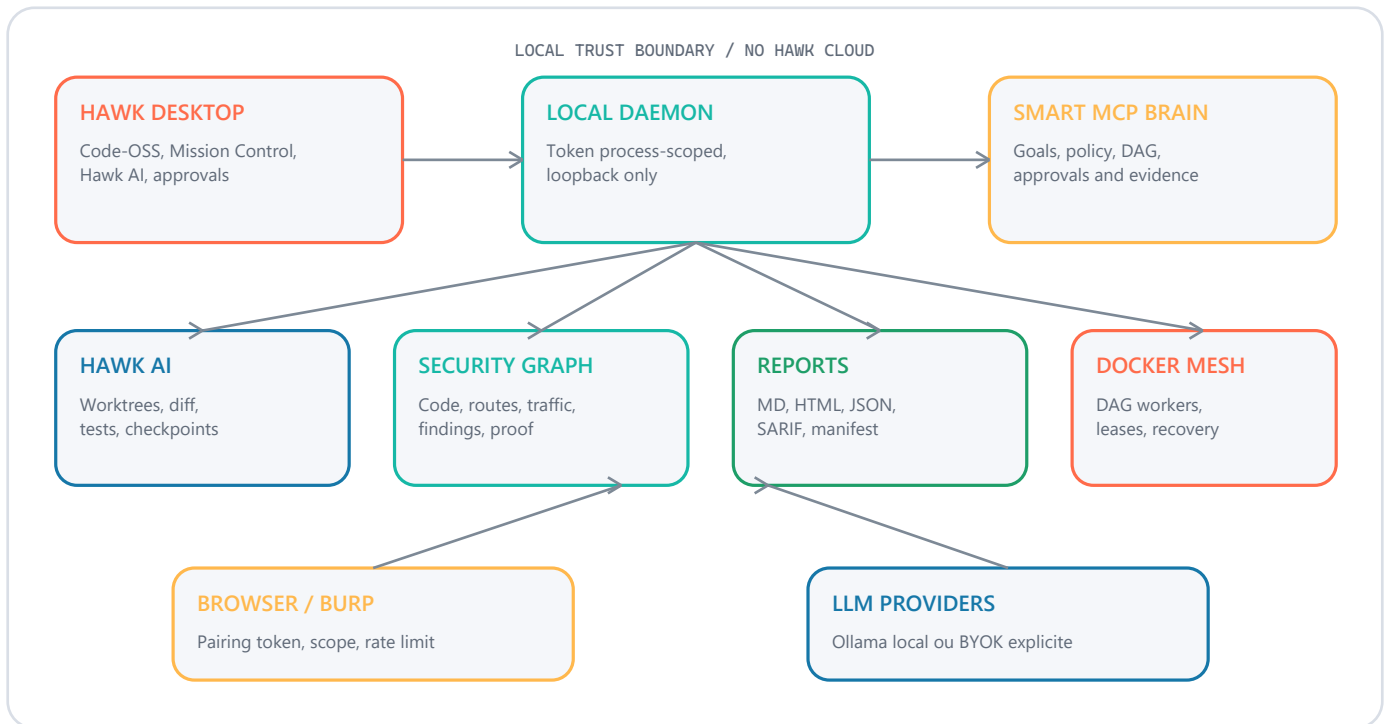


Hawk dans le shell Code-OSS brandé: Explorer, onglet Mission Control, barre Hawk et thème Obsidian.



03 / ARCHITECTURE

3. Architecture et circulation des données



Séquence normale

- L'extension démarre le daemon Hawk pour le workspace actif.
- Le daemon écoute uniquement sur loopback et exige un token aléatoire détenu par l'extension host.
- Le webview ne reçoit jamais ce token: l'extension joue le rôle de proxy.
- Mission Control demande inventaire, findings, trafic, graph, health et reproductions.
- Hawk AI crée un worktree temporaire et lance un worker séparé avec des outils fichier bornés.
- Smart MCP compile les objectifs en GoalSpec, policy et DAG avant toute exécution.
- Docker exécute seulement les tâches approuvées dans des containers isolés.
- Les sorties rejoignent ProofGraph, les rapports, les sessions et les historiques locaux.



Composants du dépôt

Dossier	Responsabilité
src/agent	Boucle agent, planner, événements, mentions, prompt système et assainissement.
src/llm	Clients Ollama, OpenAI, Anthropic, Gemini, Groq, Kimi, DeepSeek, OpenRouter et routage.
src/tools	Fichiers, shell, HTTP, recherche, MCP, browser capture, coverage et findings.
src/ide	Daemon, sessions AI, index, Next Edit, graph, scans, rapports, MCP, Docker, scheduler.
src/browser	Bridge loopback et MCP de capture Browser.
src/ui	Interface terminal Hawk CLI basée sur Ink/React.
extensions/hawk-security-ide	Extension Code-OSS/VS Code, Mission Control, Hawk AI, debug et updater.
integrations/browser	Extension Chromium Hawk Live Capture.
integrations/burp	Extension Java 21 Montoya pour Burp Suite.
desktop	Préparation Code-OSS, branding, installateurs, MSI, AppImage, deb et signature.
docker/hawk-worker	Image de base locale pour agents et reproductions.
docs	Contrats d'architecture, sécurité, MCP, reproduction, release et audit.



4. Installation, lancement et premier démarrage

Pré-requis de développement

- Windows ou Linux x64; Node.js 20 ou supérieur; Git.
- Docker Desktop uniquement pour les reproductions et workers isolés.
- Java 21 uniquement pour construire le companion Burp.
- Ollama facultatif si un provider BYOK est déjà configuré.

Construction depuis le dépôt

```
npm install
npm run build
npm run check:extension
npm run build:extension
npm run ci
```

Lancer le daemon seul

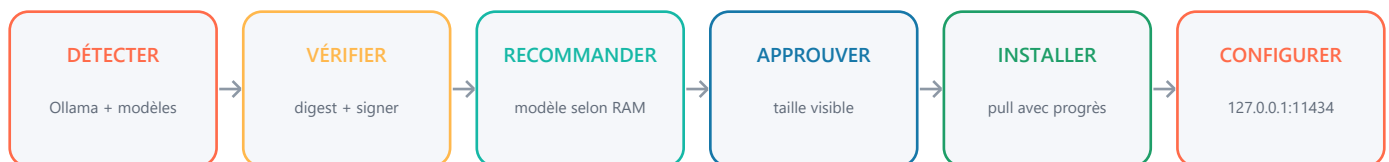
```
npm run dev:ide-daemon -- --workspace C:\chemin\vers\projet

# Le daemon imprime une URL loopback et un token de processus.
# Chaque requête doit porter: X-Hawk-Token: <token>
```

Préparer le worker Docker local

```
docker build -t hawk-worker:local docker/hawk-worker
```

Premier démarrage Local AI



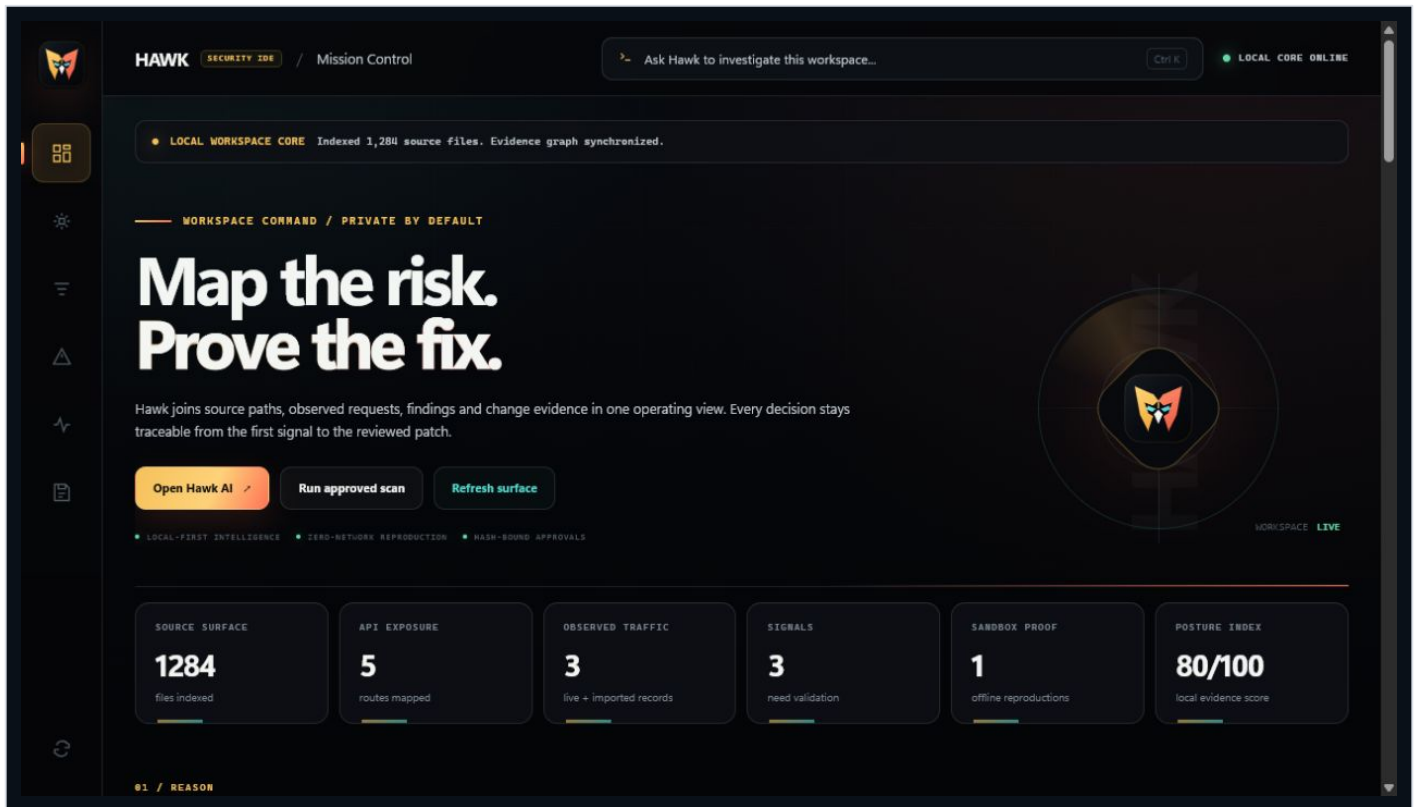
Chaque transition sensible reste visible, bornée et validée par l'opérateur.

Important

L'installation Ollama ne signifie pas qu'un modèle est téléchargé sans permission. Hawk affiche le modèle recommandé et sa taille puis attend une approbation explicite.

05 / UI

5. Mission Control - lecture de l'interface



Mission Control: état du core local, commande Hawk AI, surface de risque et indicateurs.

Les six indicateurs

Indicateur	Interprétation
Source surface	Nombre de fichiers source indexés.
API exposure	Routes API statiquement cartographiées.
Observed traffic	Requêtes live ou importées disponibles pour corrélation.
Signals	Détections qui nécessitent encore une validation.
Sandbox proof	Reproductions offline terminées.
Posture index	Score local de complétude de preuve, pas un score absolu de sécurité.



Navigation latérale

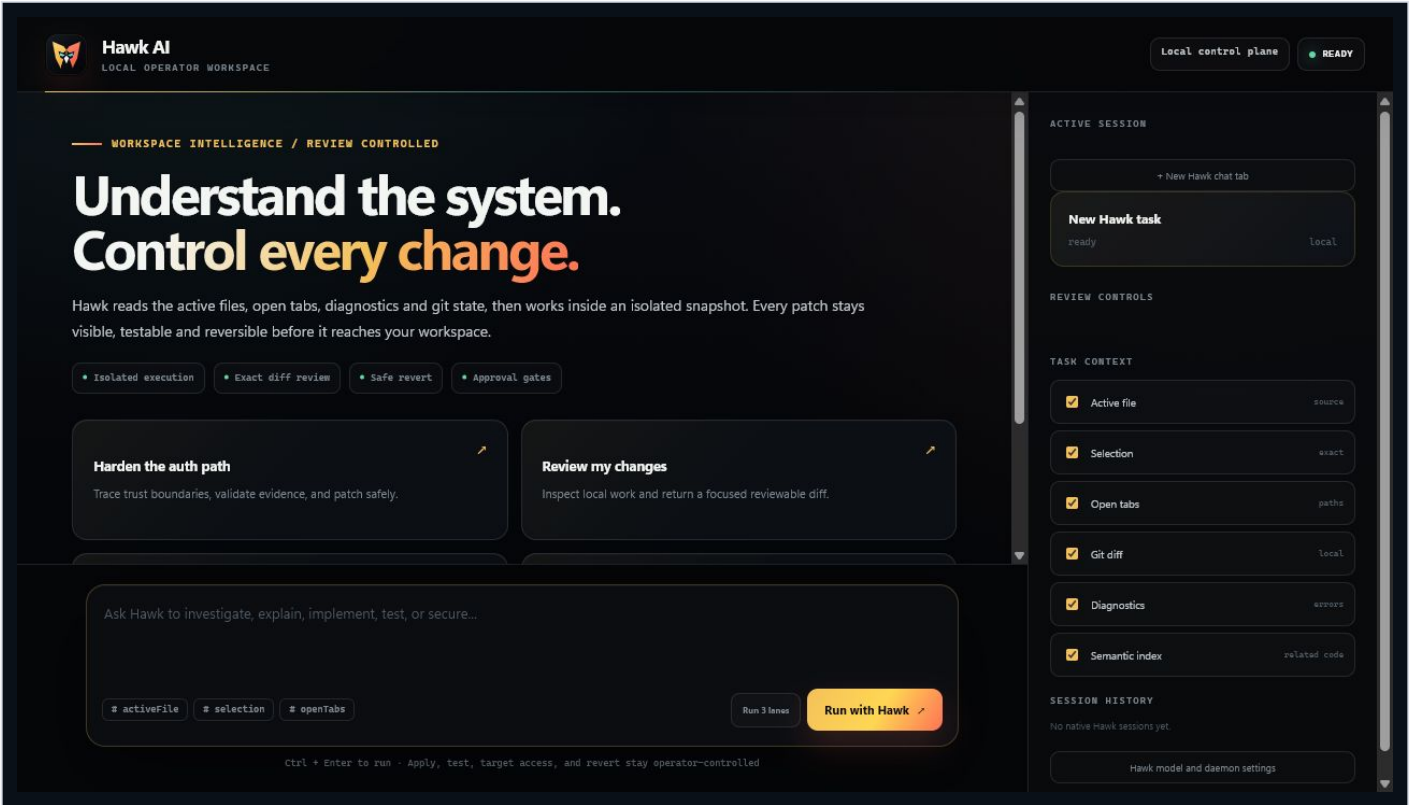
Contrôle	Action
Overview	Retour au résumé, aux KPI et au statut local.
Hawk AI	Défile vers le composeur AI et le Coding Core.
Attack surface	Défile vers la carte code <-> requêtes et l'inventaire de routes.
Findings	Défile vers la file de signaux priorités.
Traffic	Défile vers la timeline et les routes observées.
Evidence & MCP	Défile vers les preuves, la posture et le mesh MCP.
Refresh local activity	Recharge inventaire, findings, trafic, health, graph et reproductions.

06 / BOUTONS

6. Tous les boutons de Mission Control

Bouton / contrôle	Ce qu'il fait	Garde-fou
Open Hawk AI	Ouvre le panneau Hawk AI; un prompt de Mission Control peut être pré-rempli.	Workspace trusted.
Run approved scan	Choisit un template, affiche le plan/hash et demande l'approbation exacte.	Approbation modale.
Refresh surface	Réindexe les routes et fichiers du workspace.	Local, passif.
Review auth boundary	Pré-remplit une analyse de la frontière d'authentification.	Ouvre Hawk AI.
Trace request to source	Pré-remplit une corrélation trafic -> route -> source.	Ouvre Hawk AI.
Plan secure fixes	Pré-remplit un plan de remédiation basé sur les preuves.	Ouvre Hawk AI.
Generate threat model	Pré-remplit une demande de threat model du workspace.	Ouvre Hawk AI.
Flèche du compositeur	Envoie le texte saisi vers le panneau Hawk AI.	Aucun test actif implicite.
Set up local AI	Démarre l'assistant Ollama / modèle local.	Téléchargement approuvé.
Semantic search	Demande une recherche naturelle dans l'index local.	Aucun upload par défaut.
Rebuild index	Reconstruit l'index AST/type persistant.	Local.
Run benchmark	Mesure index, recherche, latence Hawk Tab et mémoire.	Gates de performance.
Check updates	Interroge le feed GitHub privé et vérifie l'installateur.	Hash + signature + approbation.
Import HAR	Importe un fichier HAR et conserve seulement l'inventaire redigé.	Pas de replay, taille bornée.
Pair live capture	Copie URL + token du bridge Browser/Burp.	Token distinct et loopback.
Replay	Choisit une requête capturée, demande 2 à 8 identities, prépare le plan/hash et exécute après second approval.	Credentials cachées, pas de redirects, rate limit, bodies jamais retournés.
Audit code / Run local audit	Exécute les cinq règles passives sur le texte source.	Signal, pas verdict.
Refresh	Recharge les données de la section Observe.	Local.
Pair	Même action de pairing depuis la carte Traffic pulse.	Capture désactivée avant opt-in.
Sync GitHub health	Récupère un health.json déjà configuré.	Token en secret storage.
Configure sync	Configure URL GitHub Raw/Contents et token facultatif.	Domaines GitHub validés.
Build evidence pack	Produit MD, HTML, JSON, SARIF et manifest SHA-256.	Approbation modale.
Plan governed mission	Crée objectif, profil, policy, DAG et rapport lisible.	Planifie sans exécuter.
Copy MCP config	Copie la configuration du MCP Hawk embarqué.	Serveur local/stdio.
Import health	Importe un health.json local de 5 MB maximum.	Résumé assaini seulement.
Open source	Ouvre le fichier et la ligne du signal.	Navigation locale.
Reproduce	Crée et approuve un plan de reproduction Docker offline.	Plan hash + Docker isolé.
Retest	Relance le détecteur sur la même source après correction.	Ne prouve pas l'impact.
Route / Graph node	Ouvre le fichier source associé au noeud.	Chemin workspace validé.
Repository posture card	Ouvre l'URL HTTPS du dépôt issue du health.json.	HTTPS uniquement.

7. Hawk AI - le workspace d'ingénierie

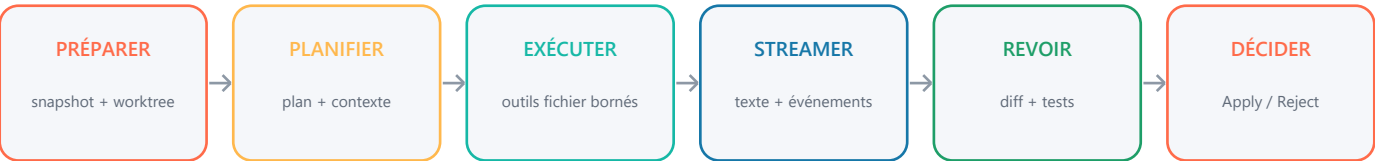


Nouvelle tâche Hawk AI: starters, compositeur, contexte sélectionnable, sessions et état du control plane.

Contexte sélectionnable

Option de contexte	Contenu
Active file	Le fichier actuellement actif.
Selection	La sélection exacte de l'éditeur.
Open tabs	Les chemins et contenus bornés des onglets utiles.
Git diff	Les changements locaux non encore commités.
Diagnostics	Erreurs et warnings de l'éditeur.
Semantic index	Régions de code locales classées par pertinence.

Cycle d'une session



Chaque transition sensible reste visible, bornée et validée par l'opérateur.

Le worker AI n'a pas de shell, navigateur, HTTP ou accès cible illimité. Il reçoit des outils fichier bornés au worktree; les workflows de sécurité sensibles passent par Smart MCP.

 **Hawk AI**
LOCAL OPERATOR WORKSPACE

openrouter · anthropic/claude-sonnet

REVIEW READY

TOOL COMPLETED · FILEEDITTOOL 1s

Updated the policy guard and added a tenant-bound ownership check.

H

I found a cross-tenant authorization gap in the role update path. The patch now binds the requested user to the active organization before evaluating role permissions, and the regression test covers the denied cross-tenant case.

REVIEW READY

Diff ready: 3 files, +46 -13.

Exact patch ready for your review

Nothing has touched your real workspace. Preview the diff, run the approved gates, then Apply or Reject.

3 files

+46

-13

4.7 KB

Ask Hawk to refine this patch, explain a change, or add a focused fix...

activeFile

selection

openTabs

Run 3 times

Run with Hawk

Ctrl + Enter to run · Apply, test, target access, and revert stay operator-controlled

ACTIVE SESSION

+ New Hawk chat tab

Harden organization role authorization

review ready now

REVIEW CONTROLS

Preview exact diff

Run gates

Apply

Reject

Type check

passed

TASK CONTEXT

☒ Active file source

☒ Selection exact

☒ Open tabs paths

☒ Git diff local

☒ Diagnostics errors

Session prête à être revue: événements, résumé du patch, résultats de gates et contrôles Apply/Reject.

Usage autorisé uniquement - données et décisions locales

Page 15

8. Tous les contrôles de Hawk AI

Contrôle	Comportement exact
Harden the auth path	Starter: trace les frontières de confiance et demande un patch minimal.
Review my changes	Starter: inspecte le git diff courant et corrige les régressions confirmées.
Fix the active failure	Starter: utilise diagnostics/erreurs pour diagnostiquer puis corriger.
Trace source to request	Starter: relie une route observée au chemin source et aux décisions d'autorisation.
Run with Hawk	Crée une session isolée avec le prompt et les contextes cochés.
Run 3 lanes	Lance trois worktrees: architecture, implementation et vérification.
+ New Hawk chat tab	Crée une nouvelle conversation sans détruire l'historique.
Session history item	Rouvre une session durable et recharge événements, état et diff.
Preview exact diff	Montre le patch complet dans le drawer et l'éditeur diff.
Run gates	Propose uniquement typecheck, lint, test et build détectés; approval requis.
Apply	Applique le hash exact après contrôle du drift et des gates.
Reject	Supprime le worktree; le workspace réel reste intact.
Revert applied patch	Inverse le patch seulement si les fichiers correspondent encore au post-apply hash.
Save checkpoint	Enregistre le patch isolé vérifié par SHA-256.
Restore checkpoint	Restaure uniquement le worktree temporaire et régénère le diff.
Open isolated terminal	Ouvre un terminal dans le worktree de revue, pas dans le workspace réel.
Smart Synthesis - best of lanes	Compare les lanes et produit une session de synthèse review-gated.
Pause and preserve task	Arrête le worker et conserve worktree + mémoire pour reprise.
Resume recovered task	Reprend une tâche restaurée sans recréer le contexte.
Stop running task	Interrompt worker ou gate en cours; ne valide aucun résultat.
Hawk model and daemon settings	Ouvre les réglages provider, modèle, base URL, Tab, index, debug et update.
Close preview	Ferme le drawer du diff sans modifier le patch.

Événements visibles dans le flux

- Identité du provider et du modèle.
- Plan de travail.
- Tool call et tool result.
- Texte assistant en streaming.
- Sortie des gates approuvées.
- Diff ready, erreurs, pause, reprise et fin.

Règle principale

Un modèle peut proposer et préparer un patch; il ne peut pas décider seul de l'appliquer au workspace réel.

9. Hawk Coding Core

Fonction	Comment elle marche
Hawk Tab	Complétion inline privée avec prefix, suffix et symboles reliés.
Next Edit	Prédiction multilignes structurée; vieux texte doit correspondre exactement au suffix courant.
Cache	Réutilise requêtes exactes, appels concurrents et insertion partiellement tapée; TTL et taille bornés.
Scorecard	Validité, feedback operator, cache hits, p50/p95; aucun code ou prompt persistant.
Semantic Index	Index persistant, incrémental, AST/type pour TS/JS, parsing structurel pour autres langages.
Embeddings	Reranking facultatif via Ollama loopback; fallback lexical si indisponible.
Model Router	Route principale + 8 fallbacks BYOK fast/reasoning/security/general.
Benchmark	Index time, search p50/p95, Hawk Tab latency, heap/RSS et gates de mémoire.
Debugger Agent	Capture DAP, redaction, diagnose/edit/test/fix borné; Apply reste manuel.
AST Semantic Merge	Transplante fichiers/symboles compatibles; conflits explicites au lieu de concaténer les patches.

Contrat mémoire

Le benchmark d'index exige moins de 500 MiB de RSS pic. Les gros fichiers, chunks résidents, données persistées et embeddings possèdent des plafonds. Une reconstruction de l'index invalide les caches qui dépendent de son identité.

Debugger Agent



Chaque transition sensible reste visible, bornée et validée par l'opérateur.



10. Modèles LLM et Local AI

Backend	Utilisation
ollama	Modèles locaux via API loopback; option recommandée pour usage personnel privé.
lmstudio	Serveur local compatible OpenAI.
openai	API OpenAI avec clé locale/BYOK.
openai-compat	Endpoint compatible OpenAI configuré par l'utilisateur.
anthropic	Claude via clé locale/BYOK.
gemini	Google Gemini via clé locale/BYOK.
groq	Provider Groq via clé locale/BYOK.
openrouter	Routage de modèles OpenRouter via clé locale/BYOK.
deepseek	API DeepSeek via clé locale/BYOK.
kimi	API Kimi via clé locale/BYOK.

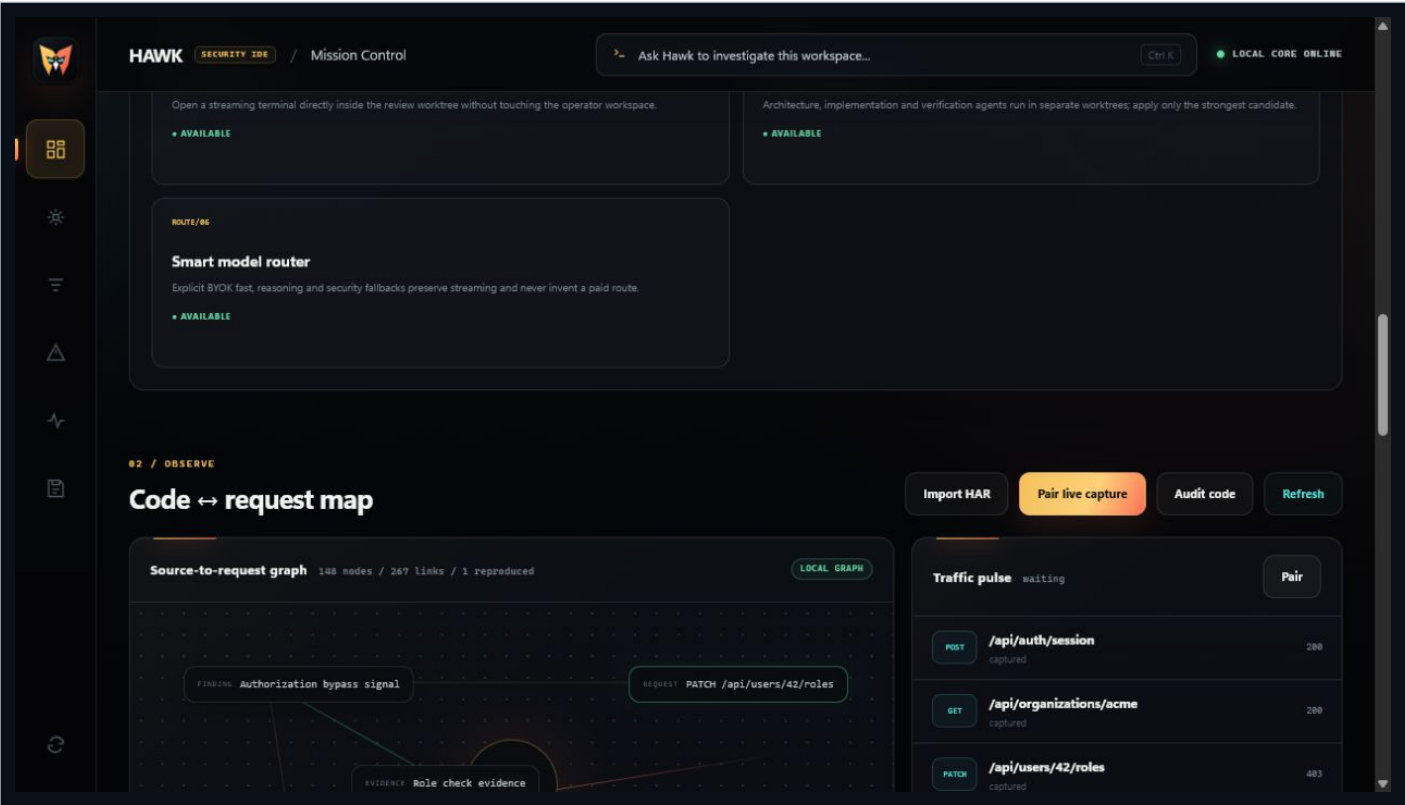
Politique des clés

- Les clés restent sur la machine et peuvent venir de variables d'environnement.
- Un fallback sans credential reste dormant.
- Un fallback n'est essayé qu'avant tout chunk streamé par la route primaire.
- L'accès hosted est autorisé seulement quand la policy de la tâche le permet.
- Hawk n'invente pas de route payante et n'a pas de système de facturation.

Installation Ollama contrôlée

- Résolution de la release officielle ollama/ollama.
- Acceptation uniquement de OllamaSetup.exe depuis le chemin officiel.
- Vérification digest SHA-256, taille et hôte final.
- Vérification Authenticode du signer Ollama sur Windows.
- Recommandation d'un modèle Qwen coding selon la RAM et affichage de la taille.
- Pull après approbation, puis configuration du provider et restart daemon.

11. Workflow sécurité de bout en bout



Code <-> request map: source, route, requête observée, finding et evidence partagent le même graphe.

Indexation et règles passives

Rule ID	Sévérité	Signal
hardcoded-secret	HIGH	Credential-like value intégrée au code; la valeur est redigée.
tls-verification-disabled	HIGH	rejectUnauthorized: false.
dynamic-code-execution	HIGH	Appel eval(...).
sql-template-interpolation	HIGH	Interpolation template dans une fonction query/execute/raw.
wildcard-cors-credentials	MEDIUM	Origin wildcard combiné avec credentials: true.

Lecture correcte

Ces règles lisent le texte source. Elles ne démarrent pas l'application et ne contactent aucune cible. Le résultat est un signal, pas une confirmation d'exploitabilité.

Templates de scan

Template	Réseau	Contenu
passive-workspace	Offline	Routes + règles passives + trafic déjà importé + rapport local.
runtime-observe	Captured-only	Corrèle jusqu'à 1 500 requêtes capturées sans génération/replay.
release-gate	Offline	Snapshot pré-release depuis code, trafic local et health importé.

12 / PROOF

12. Findings, reproduction et retest

The screenshot shows the HAWK Security IDE interface. At the top, there's a header with the HAWK logo, 'SECURITY IDE', and 'Mission Control'. A search bar says 'Ask Hawk to investigate this workspace...'. On the right, it says 'LOCAL CORE ONLINE'. The main area is titled '03 / DECIDE' and 'Prioritized security queue'. It contains an 'Investigation queue' ranked by severity and evidence. The queue lists several findings: 'Automatic sandbox reproduction' (LAB), 'Authorization decision can be bypassed' (HIGH), 'Webhook signature uses non-constant comparison' (MEDIUM), and 'Verbose error metadata reaches client' (LOW). Each finding has buttons for 'Open source', 'Reproduce', and 'Retest'. At the bottom, there's a section '04 / PROVE' titled 'Evidence, policy & agent mesh' with buttons for 'Sync GitHub health', 'Configure sync', 'Build evidence pack', 'Plan governed mission', and 'Copy MCP config'.

File priorisée: sévérité, source, état de reproduction et actions Open source / Reproduit / Retest.

Cycle de vie d'un finding



Chaque transition sensible reste visible, bornée et validée par l'opérateur.

Sandbox de reproduction

- Plan valable dix minutes et lié au finding, fichier, ligne, source SHA-256, image et limites.
- Baseline: valide la source et produit un digest borné.
- Negative control: vérifie que la règle reste négative sur une valeur sûre.
- Reproduction: applique le détecteur à la source exacte sans imprimer le secret.
- Container: réseau none, rootfs/workspace read-only, non-root, capabilities drop, no-new-privileges.
- Limites usuelles: 0.5 CPU, 256 MB RAM, 30 secondes par gate, 32 MB d'artifacts.
- Même avec trois gates passées, promotedToVerified reste false avant vérification indépendante.



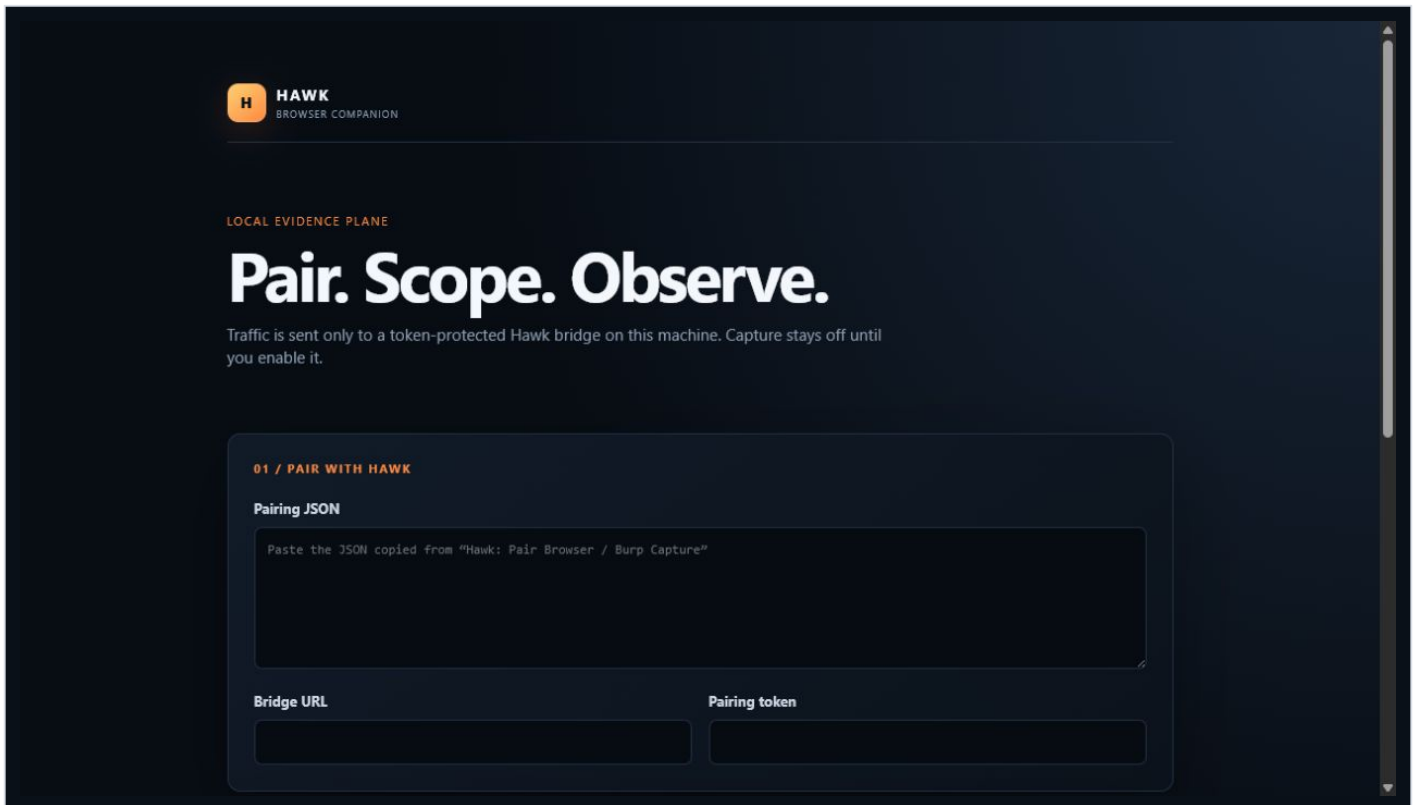
Neuf gates de vérification

Gate	Nom	Condition
01	Baseline / control	Une observation de référence existe.
02	Reproduction	Le comportement est reproduit avec succès.
03	Indépendance	Une deuxième voie de reproduction confirme le signal.
04	Identité	Le profil et l'identité de test sont valides.
05	Impact	L'impact réel est démontré, pas seulement supposé.
06	Scope	La cible, l'action et l'identité sont explicitement autorisées.
07	Side effects	Aucun effet secondaire dangereux ou hors portée.
08	Redaction	Secrets, cookies et credentials sont retirés de la preuve.
09	Evidence URI	Au moins une preuve durable est adressée par URI.

Pourquoi neuf gates?

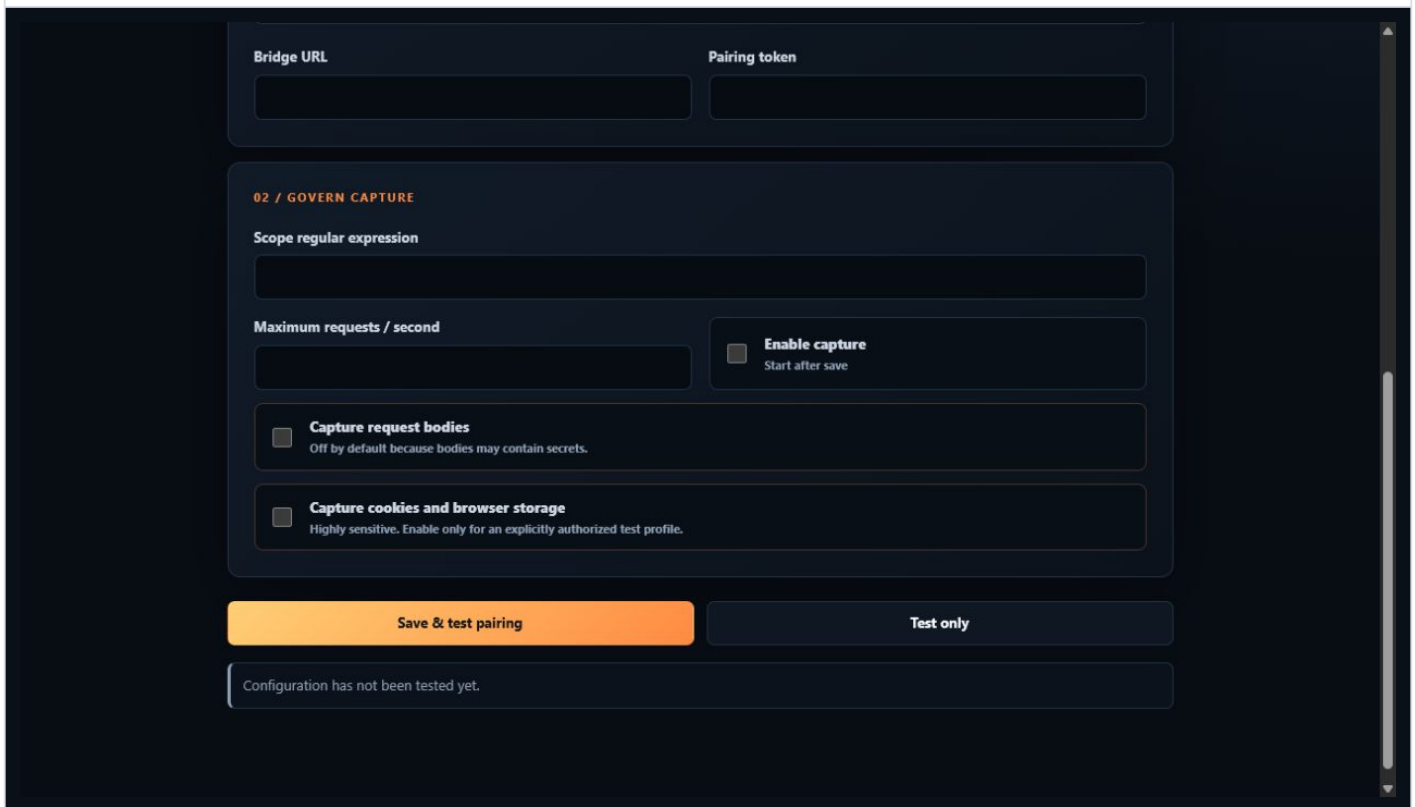
Hawk sépare volontairement la détection, la reproduction et la vérification. Cette séparation réduit les faux positifs et empêche une preuve technique isolée de devenir un verdict sans scope, identité ou impact.

13. Browser et Burp Companions



The screenshot shows the Hawk Browser Companion interface. At the top, there's a logo with 'H' and 'HAWK BROWSER COMPANION'. Below it, the text 'LOCAL EVIDENCE PLANE' is visible. The main heading is 'Pair. Scope. Observe.' followed by a subtext: 'Traffic is sent only to a token-protected Hawk bridge on this machine. Capture stays off until you enable it.' The interface is divided into sections. The first section is '01 / PAIR WITH HAWK'. It contains a 'Pairing JSON' field with a placeholder text: 'Paste the JSON copied from "Hawk: Pair Browser / Burp Capture"'. Below this are two input fields: 'Bridge URL' and 'Pairing token'.

Browser Companion: collage du pairing JSON généré par Hawk et connexion au bridge local.



The screenshot shows the Hawk Browser Companion interface, specifically the '02 / GOVERN CAPTURE' section. It features several configuration options: 'Bridge URL' and 'Pairing token' input fields at the top. Below them is the 'Scope regular expression' input field. The 'Maximum requests / second' is set to 10. There are three checkboxes: 'Enable capture' (checked), 'Capture request bodies' (unchecked), and 'Capture cookies and browser storage' (unchecked). The 'Enable capture' checkbox has a subtext 'Start after save'. The 'Capture request bodies' checkbox has a subtext 'Off by default because bodies may contain secrets.' The 'Capture cookies and browser storage' checkbox has a subtext 'Highly sensitive. Enable only for an explicitly authorized test profile.' At the bottom, there are two buttons: 'Save & test pairing' (orange) and 'Test only' (grey). A status message at the very bottom says 'Configuration has not been tested yet.'

Capture gouvernée: regex de scope, requêtes/seconde, activation, bodies et browser storage séparés.



Browser Companion

- Capture désactivée par défaut.
- Pairing JSON contient seulement URL loopback et token local.
- Regex de scope obligatoire; rate limit de 1 à 500 requêtes/seconde.
- Métadonnées webRequest et activité Fetch/XHR/WebSocket.
- Capture des bodies et du storage/cookies sont deux opt-ins séparés et sensibles.
- Aucun analytics, publicité ou cloud capture Hawk.

Burp Companion

- Extension Java 21 basée sur l'API Montoya.
- Onglet Hawk pour pairing, scope, activation et rate limit.
- Option par défaut: capturer uniquement les requêtes déjà dans le scope Burp.
- Headers de credentials redigés, queue bornée et forwarding asynchrone.
- Publication BApp Store préparée mais soumise au compte et à la revue PortSwigger.

Identity Replay gouverné

- Rejoue une requête capturée uniquement vers son host et son port exacts, avec 2 à 8 identities choisies par l'opérateur.
- Le plan est hash-bound et demande une seconde approbation avant l'exécution; débit borné de 0.1 à 5 requêtes/seconde.
- Redirects désactivés, body d'entrée plafonné à 64 KiB, préfixe de réponse plafonné à 128 KiB.
- Hawk ne conserve ni credential ni body de réponse et ne retourne que des fingerprints, statuts et tailles bornées.
- Les différences entre identities sont des pistes à examiner, jamais un verdict automatique d'autorisation.



14. Evidence Builder et Security Graph

Artifact	Rôle
report.md	Lecture analyste et revue Git.
report.html	Présentation portable dans un navigateur.
evidence.json	Automatisation structurée.
findings.sarif	Interopérabilité code scanning.
manifest.json	SHA-256 et taille de chaque artifact.

Noeuds du graphe

Repository, Commit, File, Symbol, Route, Identity, Request, Response, Finding, Evidence, Patch, Test, Run, Agent, Tool et Model. Chaque edge peut porter provenance et confiance.

Exemples de navigation

```
HTTP request -> route -> source code -> patch -> regression test
static signal -> sandbox reproduction -> verification gates -> retest
agent session -> patch hash -> test gates -> Apply/Revert decision
```

Principe

Une requête corrélée à un finding fournit du contexte. Cette relation ne transforme jamais automatiquement le finding en vulnérabilité vérifiée.

15. Smart MCP Brain



Chaque transition sensible reste visible, bornée et validée par l'opérateur.

Contrat de goal

- Repositories, hosts, routes et identities autorisés.
- Actions autorisées et interdites.
- Budgets de parallélisme, temps, tokens, coût et request rate.
- Data policy local-only ou hosted.
- Modèles préférés par rôle et classe.
- Mode d'approbation, rétention et critères de succès.

Autorité

Une approbation ne peut pas réparer une autorité absente. Si l'action n'est pas dans allowed_actions, la policy la refuse même si le prompt demande le contraire.

Outils Smart MCP

Outil	Fonction
hawk_capabilities_search	Trouve les capacités pertinentes sans charger tous les schemas.
hawk_context_snapshot	Capture un contexte passif borné du workspace.
hawk_plan_create	Compile objectif -> GoalSpec, policy, DAG et hash.
hawk_plan_approve	Crée une approval courte liée au goal, plan id et hash.
hawk_run_start	Démarre un run durable après revalidation policy/hash.
hawk_run_observe	Lit état, événements et artifacts d'un run.
hawk_run_control	Pause, resume ou cancel selon l'état autorisé.
hawk_run_execute_task	Expose le moteur via le protocole MCP Tasks durable.
hawk_evidence_verify	Applique les neuf gates et empêche une promotion injustifiée.
hawk_memory	Mémoire run/project/organization gouvernée par preuve.
hawk_mcp_security_audit	Sentinel: injection, poisoning, secret, drift/rug pull, egress.
hawk_patch_tournament	Prépare lanes, tests, juges et décision humaine sans éditer.
hawk_eval_lab	Compare Hawk à une baseline avec modèle/budgets identiques.
hawk_a2a_bridge	Import/export local d'enveloppes de tâches A2A.
hawk_mission_control	Données et contrôle de l'app MCP sandboxée.



16. MCP - ressources, prompts et outils techniques

Ressources

URI	Contenu
hawk://workspace/graph	Graphe de sécurité du workspace.
hawk://run/{runId}/events	Chaine d'événements du run.
hawk://run/{runId}/artifact/{nodeId}	Artifact structuré d'un node.
hawk://finding/{findingId}/proof	Preuve et gates d'un finding.
hawk://policy/{planId}	Policy et contrat du plan.
hawk://interop/a2a-profile	Profil d'interopérabilité A2A.
ui://hawk/mission-control.html	MCP App sandboxée sans egress.

Prompts

Prompt	But
hawk_secure_pr_review	Revue de PR sécurisée et bornée.
hawk_idor_matrix	Matrice IDOR/BOLA explicitement scoped.
hawk_verify_finding	Collecte les gates manquantes avant verify.
hawk_fix_and_retest	Tournoi patch + tests + décision de retest.



Outils MCP IDE et Docker

Outil	Fonction
ide_route_inventory	Inventaire passif des routes.
ide_static_audit	Audit texte et IDs de findings.
hawk_supply_chain_health	Résumé health.json importé.
hawk_reproduction_plan	Plan de reproduction expirant.
hawk_reproduction_execute	Exécution du plan hash-approved.
hawk_reproductions_list	Historique des tentatives.
hawk_parallel_estimate	Critical path et plancher théorique.
hawk_parallel_runtime	État du Docker runtime.
hawk_scheduler_status	Instances, capacités, leases et décisions.
hawk_docker_desktop_status	État Docker Desktop.
hawk_docker_desktop_start	Demande start après approval.
hawk_docker_desktop_stop	Demande stop avec avertissement containers.
hawk_parallel_start	Valide et démarre un task graph.
hawk_parallel_status	Progression et logs bornés.
hawk_parallel_runs	Liste runs actifs/restaurés.
hawk_parallel_cancel	Annule pending et supprime les workers actifs.

Outils MCP Browser

Outil	Fonction
browser_capture_status	Compte requests/endpoints/snapshots et dernière activité.
browser_capture_endpoints	Liste endpoints uniques et noms de paramètres.
browser_capture_requests	Liste les requêtes récentes et leurs IDs.
browser_capture_get	Détail borné d'une requête capturée.
browser_capture_snapshot	Dernier snapshot explicitement capturé.
browser_capture_clear	Efface le store mémoire après permission.



17. Docker Worker Mesh et récupération

Capacité et scheduling

- 64 tâches maximum par run; jusqu'à 32 workers actifs.
- DAG dependency-aware: les dépendances échouées bloquent les étapes downstream.
- Priorité et critical path pour classer les tâches prêtes.
- Placement selon capabilities, CPU/RAM, affinité, charge, failure rate et durée observée.
- Stratégies balanced, latency ou throughput.
- Chaque assignment reçoit un lease renouvelé par heartbeat.
- Une retry autorisée peut être réassignée à une autre instance saine.

Isolation de chaque worker

Contrôle	Implémentation
Image	--pull=never, tag résolu vers identité sha256 immuable.
Workspace	Monté read-only sur /workspace.
Output	tmpfs unique et quota-limité, copié après terminaison.
Process	UID/GID non-root, capabilities drop, no-new-privileges.
Filesystem	Root filesystem read-only.
Network	none par défaut; restricted passe par le proxy Hawk authentifié et allowlist host/port; env exige approved_external_access.
Resources	CPU, RAM, PID, open files, timeout, retry et plafonds globaux.
Logs	Taille bornée; artifacts séparés par task.

Crash, restart et network failure

- run.json et spec.json sont écrits atomiquement sous .hawk/orchestrations/.
- Après restart, Hawk restaure l'historique et tente de se rattacher au container retenu.
- Exit code, logs et artifacts sont récupérés avant de continuer le DAG.
- Une erreur runtime comme ECONNRESET devient un échec borné et retryable, jamais un run bloqué.
- Une erreur pendant le reattach peut relancer seulement si le budget retry le permet.
- Les sessions AI background récupèrent leur worktree et peuvent auto-resume.
- Les paths .hawk, symlinks/junctions et artifacts spéciaux sont refusés.

Chaos tests

La suite dédiée couvre worker crash + control-plane restart, network ECONNRESET, échec de recovery et reprise automatique d'un background agent. Commande: `npm run test:chaos`.



18 / SETTINGS

18. Réglages Hawk - catalogue complet

Réglage	Type	Défaut	Description
hawk.daemonPath	string		Absolute path to hawk-ide-daemon. Leave empty to use the executable on PATH.
hawk.hawkHealthUrl	string		GitHub raw or Contents API URL for a Hawk health.json report. Optional token is held only in Hawk's encrypted local secret storage.
hawk.preferredProvider	string		Preferred LLM provider name. API keys stay on this machine.
hawk.preferredModel	string		Preferred model ID. API keys stay on this machine.
hawk.preferredBaseUrl	string		Optional local or provider-compatible base URL used by Hawk AI and Hawk Tab.
hawk.localAI.offerSetupOnFirstRun	boolean	true	Offer the verified Ollama and local coding model setup when Hawk has no configured provider.
hawk.openMissionControlOnStartup	boolean	true	Open the Hawk Mission Control editor whenever a Hawk window starts.
hawk.tab.enabled	boolean	true	Enable private Hawk inline code completion.
hawk.tab.debounceMs	number	180	Delay before Hawk requests an inline completion. min 80; max 2000
hawk.tab.maxPrefixChars	number	12000	Maximum code-before-cursor context sent to the configured model. min 2000; max 40000
hawk.tab.maxSuffixChars	number	4000	Maximum code-after-cursor context sent to the configured model. min 500; max 12000
hawk.tab.editPrediction.enabled	boolean	true	Predict the next safe multiline edit from recent changes, diagnostics, and repository context.
hawk.tab.editPrediction.minConfidence	number	0.55	Discard model-proposed multiline edits below this confidence threshold. min 0.3; max 0.95
hawk.tab.editPrediction.cacheEnabled	boolean	true	Reuse exact and partially typed Next Edit predictions in a bounded in-memory cache. Applies after the local control plane restarts.
hawk.tab.editPrediction.cacheTtlSeconds	number	120	Lifetime of an in-memory Next Edit cache entry. Applies after the local control plane restarts. min 5; max 600
hawk.tab.editPrediction.cacheMaxEntries	number	256	Maximum in-memory Next Edit entries. Source text is never written to the persistent model scorecard. min 32; max 512
hawk.index.embeddings.enabled	boolean	false	Add optional local Ollama embeddings to Hawk's persistent AST/type index. Source remains on this machine.
hawk.index.embeddings.model	string	embeddinggemma	Local Ollama embedding model used by Hawk hybrid search.
hawk.index.embeddings.baseUrl	string	http://127.0.0.1:11434	Loopback-only Ollama URL for optional Hawk embeddings.
hawk.debug.autoFix.maxAttempts	number	3	Maximum approved diagnose/edit/test retries in one isolated Hawk Debug Agent loop. Apply always remains manual. min 1; max 6
hawk.reproduction.image	string	hawk-worker:local	Existing local Docker image for Hawk's offline, zero-network vulnerability-signal reproduction sandbox.
hawk.updates.checkOnStartup	boolean	true	Check the private Hawk GitHub release feed after startup.
hawk.updates.channel	string	stable	Hawk release channel. enum: stable beta
hawk.updates.expectedPublisher	string		Optional Windows certificate subject pin. Every Windows update always requires a valid, Windows-trusted Authenticode signature.

19. Command Palette et raccourcis

ID	Titre affiché	Rôle
hawk.startDaemon	Hawk: Start Local Control Plane	Démarre le control plane loopback.
hawk.indexWorkspace	Hawk: Index Security Surface	Indexe fichiers, symboles et routes.
hawk.openAgent	Hawk: Open AI Agent	Ouvre Hawk AI.
hawk.runWorkspaceScan	Hawk: Run Approved Passive Workspace Scan	Lance le picker de scan gouverné.
hawk.syncHawkHealth	Hawk: Sync GitHub Health Report	Synchronise le rapport health.json configuré.
hawk.configureHawkHealthSync	Hawk: Configure GitHub Health Sync	Configure URL/token de health sync.
hawk.openSecurityDashboard	Hawk: Open Mission Control	Ouvre Mission Control en plein éditeur.
hawk.pairCapture	Hawk: Pair Browser / Burp Capture	Copie le pairing Browser/Burp.
hawk.planIdentityReplay	Hawk: Plan Governed Identity Replay	
hawk.buildEvidencePack	Hawk: Build Sanitized Evidence Pack	Construit les artefacts de preuve assainis.
hawk.planGovernedMission	Hawk: Plan Governed Smart MCP Mission	Crée un GoalSpec et un DAG reviewable.
hawk.toggleTab	Hawk: Toggle Inline Completion	Active/désactive Hawk Tab.
hawk.triggerTab	Hawk: Trigger Inline Completion	Demande immédiatement une prediction inline.
hawk.rebuildSemanticIndex	Hawk: Rebuild Semantic Workspace Index	Reconstruit l'index persistant.
hawk.searchWorkspace	Hawk: Search Workspace Semantically	Recherche naturelle dans le code.
hawk.runCodingBenchmark	Hawk: Run Coding Core Benchmark	Mesure performance et mémoire.
hawk.showEditPredictionEvaluation	Hawk: Show Next Edit Model Evaluation	Affiche la scorecard Next Edit.
hawk.clearEditPredictionCache	Hawk: Clear Next Edit Cache	Efface le cache source mémoire.
hawk.checkForUpdates	Hawk: Check for Private Release Updates	Vérifie les releases privées.
hawk.configureReleaseToken	Hawk: Configure Private Release Access	Stocke l'accès release en secret storage.
hawk.setupLocalAI	Hawk: Set Up Local AI with Ollama	Assistant Ollama + modèle local.
hawk.showLocalAIStatus	Hawk: Show Local AI Status	Affiche runtime/modèles/configuration locale.
hawk.debug.capture	Hawk: Capture Native Debug Snapshot	Capture snapshot DAP redigé.
hawk.debug.fixFailure	Hawk: Run Automatic Debug / Test / Fix Loop	Démarre la boucle debug/test/fix.
hawk.debug.stopFixLoop	Hawk: Stop Automatic Debug Loop	Arrête la boucle active.

Raccourcis

Raccourci	Action
Ctrl+Shift+H / Cmd+Shift+H	Ouvrir Hawk Mission Control.
Alt+\	Déclencher Hawk Tab / Next Edit dans l'éditeur.
Ctrl+K	Command bar Hawk dans Mission Control.
Ctrl+Enter	Envoyer une tâche depuis Hawk AI.

20 / API

20. API locale du daemon

Méthode	Endpoint	But
GET	/v1/health	Santé et version protocole.
POST	/v1/workspace/index	Index route/source.
GET	/v1/workspace/inventory	Inventaire courant.
POST	/v1/workspace/semantic-index	Build/rebuild index.
GET	/v1/workspace/semantic-index	Statut index.
PUT	/v1/workspace/semantic-index/file	Update incrémental d'un fichier.
DELETE	/v1/workspace/semantic-index/file	Suppression incrémentale.
POST	/v1/workspace/search	Recherche sémantique.
POST	/v1/ai/inline-completion	Hawk Tab insertion.
POST	/v1/ai/edit-prediction	Next Edit multiligne.
POST	/v1/ai/edit-prediction/feedback	Acceptance/rejection agrégée.
GET	/v1/ai/edit-prediction/evaluation	Scorecard modèles/cache/latence.
DELETE	/v1/ai/edit-prediction/cache	Efface cache en mémoire.
POST	/v1/diagnostics/coding-core	Benchmark Coding Core.
GET	/v1/ai/sessions	Liste sessions.
POST	/v1/ai/sessions	Crée session.
GET	/v1/ai/sessions/:id	État session.
GET	/v1/ai/sessions/:id/events	Événements après un ID.
POST	/v1/ai/sessions/:id/messages	Continue la session.
GET	/v1/ai/sessions/:id/diff	Patch exact.
POST	/v1/ai/sessions/:id/tests	Gates approuvées.
POST	/v1/ai/sessions/:id/tests/cancel	Arrête les gates.
POST	/v1/ai/sessions/:id/apply	Apply hash-bound.
POST	/v1/ai/sessions/:id/checkpoints	Crée checkpoint.
POST	/v1/ai/sessions/:id/checkpoints/restore	Restaure checkpoint.
POST	/v1/ai/sessions/:id/reject	Reject approuvé.
POST	/v1/ai/sessions/:id/revert	Revert drift-safe.
POST	/v1/ai/sessions/:id/cancel	Stop task.
POST	/v1/ai/sessions/:id/pause	Pause durable.
POST	/v1/ai/sessions/:id/resume	Resume durable.
POST	/v1/ai/batches	Crée trois lanes.
POST	/v1/ai/batches/merge	AST semantic synthesis.
GET	/v1/findings	Signaux courants.
POST	/v1/audit/static	Audit passif.
GET	/v1/security/graph	Graph ou subgraph nodeld/depth.



Méthode	Endpoint	But
GET	/v1/traffic	Traffic importé + live.
POST	/v1/traffic/import/har	Import HAR redigé.
POST	/v1/traffic/replay/plan	Planifie un identity replay lié au host/port et aux headers credentials.
POST	/v1/traffic/replay/execute	Exécute un replay après seconde approbation; retourne des fingerprints bornés.
GET	/v1/hawk/health	Résumé supply-chain local.
POST	/v1/hawk/health/import	Import health.json.
GET	/v1/scans/templates	Trois templates.
GET	/v1/scans/plan	Plan et approval hash.
POST	/v1/scans/run	Exécute scan approved.
POST	/v1/reports/evidence	Build evidence pack.
POST	/v1/missions/plan	Compile mission MCP.
GET	/v1/reproductions	Historique.
POST	/v1/findings/:id/reproduction-plan	Plan sandbox.
POST	/v1/findings/:id/reproduce	Exécution sandbox.
POST	/v1/findings/:id/retest	Retest du signal.

Protection HTTP

Loopback obligatoire, peer + Host stricts, token X-Hawk-Token, body max 5 MB, timeouts bornés, réponses no-store/nosniff/CSP/referrer-policy.



21. CLI Hawk

Flag	Effet
--backend	Choisit ollama/lmstudio/openai/openai-compatible/kimi/groq/openrouter/deepseek/gemini/anthropic.
--model	Surcharge le modèle.
--base-url	Surcharge l'endpoint.
--api-key	Clé de session; préférer secret/env.
--skills	Ajoute des dossiers de skills.
--resume	Reprend une session.
--browser	Active Browser MCP pour cette session seulement.
--burp [port]	Démarré le bridge Burp/Hawk, port 9999 par défaut.
--no-stream	Désactive le streaming si le backend le gère mal.
--yolo	Auto-approve non-sensitive tool calls; reste risqué.
--list-skills	Liste les playbooks.
--list-tools	Liste les outils disponibles.
--log	Chemin du log runtime.
--debug-session	Écrit un JSONL complet de debug.
--version / --help	Version et aide.

Commandes slash du TUI

Commande	Action
/help	Aide.
/plan	Affiche ou modifie la planification.
/clear / /reset	Nettoie ou réinitialise la session.
/exit	Quitte.
/target	Déclare la cible autorisée.
/maxsteps	Borne le nombre d'étapes.
/thinking	Configure le mode de reasoning.
/update	Met à jour selon le workflow CLI.

Outils du CLI

Lecture/édition/écriture fichier, glob, grep, shell/PowerShell, HTTP, web fetch/search, skills et payloads, coverage de tests, confirm_finding, capture Browser/Burp et outils MCP découverts. Les appels sensibles passent par permission, sauf mode YOLO pour les opérations non sensibles.



22. Données locales, sécurité et confidentialité

Chemin	Contenu
.hawk/health.json	Résumé organization health assaini.
.hawk/reports/	Scans et evidence packs.
.hawk/plans/	Rapports de missions gouvernées.
.hawk/brain/	Goals, plans, policy, approvals, memory et runs.
.hawk/orchestrations/	Snapshots Docker et artifacts par tâche.
~/.hawk/ide/workspaces//ai-sessions/	Sessions, événements, mémoire, patches et checkpoints.
~/.hawk/ide/prediction-evaluation/	Compteurs/latences agrégés sans code.

Hardening implémenté

- Loopback socket peer + Host strict contre spoofing et DNS rebinding local.
- Tokens process-scoped; capture token séparé.
- Symlinks/junctions/non-directory refusés dans la persistance .hawk.
- Artifacts Docker parcourus récursivement; symlinks et special files refusés.
- Secrets redigés dans trafic, debugger, MCP et evidence.
- Apply/Revert vérifient preimage/postimage hashes et drift.
- Images Docker résolues vers identité immutable et jamais pulled implicitement.
- Updates Windows: taille, host, SHA256SUMS, PE, Authenticode et publisher pin facultatif.
- Chaos tests pour crash, restart, network failure et agent recovery.

Politiques

[Security Policy](#) - [Responsible Use](#) - [Threat Model](#) - [External Pentest Runbook](#)

23 / RELEASE

23. Build desktop, release et auto-update

Cible	Artifact
Windows	Portable ZIP, EXE installer, MSI.
Linux	tarball, deb, AppImage.
IDE extension	VSIX.
Browser	ZIP de publication.
Burp	JAR Java 21.
Integrity	SHA-256 manifest pour les assets.

Update réel

- Canaux stable et beta.
- Tri semver des releases privées.
- Download du bon installateur Windows seulement.
- Validation redirect host, taille, PE header et SHA-256.
- Authenticode Windows obligatoire avant lancement d'un update production.
- Installation seulement après approval de l'opérateur.

Actions owner restantes

Action	Pourquoi
GitHub Actions billing	Réparer paiement/spending limit pour relancer les builds cloud.
Official v0.7.0 release	Publier les artifacts validés dans GitHub Releases après CI verte et signature.
Windows code signing	Ajouter PFX/Azure Artifact Signing et publisher pin.
Chrome Web Store	Compte développeur, item ID, OAuth et revue listing.
PortSwigger	Soumission et revue BApp Store.
External pentest	Fournir un release candidate signé à un assessor indépendant.
Real beta	Tester sur de vrais grands projets avant lancement public.

Périmètre personnel

Il n'y a volontairement ni Apple build, ni compte Hawk, ni team/RBAC, ni Stripe, ni abonnement, ni cloud sync, ni télémétrie.



24. Workflows pratiques

A. Corriger une fonctionnalité avec Hawk AI

- Ouvrir le projet et vérifier Workspace Trust.
- Ouvrir Hawk AI, choisir les contextes et décrire le résultat.
- Lire plan, tool calls et réponse streamée.
- Preview exact diff, sauvegarder un checkpoint si utile.
- Run gates, inspecter les résultats.
- Apply si le hash, le drift et les tests sont acceptables; sinon Reject ou demander une correction.
- Revert seulement si le workspace n'a pas divergé après Apply.

B. Examiner un signal de sécurité

- Index Security Surface puis Run local audit.
- Ouvrir le fichier source du signal.
- Importer HAR ou pair live capture pour ajouter du contexte runtime.
- Utiliser Code <-> request map pour relier requête, route, source et finding.
- Reproduire dans Docker si la règle est supportée et le plan exact est approuvé.
- Collecter les gates manquantes avec Smart MCP; ne promouvoir qu'après vérification.
- Corriger avec Hawk AI, tester, Apply puis Retest.
- Build evidence pack pour livrer la preuve.

C. Distribuer une tâche longue

- Découper en tâches indépendantes avec dépendances explicites.
- Estimer critical path avec hawk_parallel_estimate.
- Vérifier Docker et les capacités des instances.
- Démarrer le graph avec ressources, timeouts, retries et network none.
- Poll hawk_parallel_status au lieu de garder un appel ouvert.
- En cas de crash, laisser Hawk reattach/retry selon le budget.
- Comparer artifacts, tests et preuves avant Apply.

D. Utiliser Hawk pour bug bounty autorisé

- Déclarer la cible et la portée autorisée avant toute action active.
- Commencer par index, capture passive et cartographie.
- Utiliser Smart MCP authorized-validation seulement avec hosts explicites et rate limit.
- Ne jamais considérer un prompt ou une credential comme autorisation.
- Conserver une preuve minimale, rédigée et reproductible.

25. Validation actuelle, limites et roadmap

95 fichiers de tests passés	745 tests passés	16 tests skipped	4/4 chaos scenarios
PASS TypeScript	PASS Biome	PASS Build tsup	<500 MiB index memory gate

Limites délibérées ou restantes

- Architecture single-operator et single-workspace; pas de multi-tenant.
- Le mode restricted impose désormais un proxy Hawk authentifié avec allowlist exacte des hosts et ports; bridge reste un alias de compatibilité.
- L'index route ne remplace pas une analyse data-flow complète.
- Les règles passives couvrent cinq patterns, pas toutes les classes de vulnérabilités.
- La reproduction supporte des signaux déterministes précis, pas une exploitation générale.
- Sentinel ne rend pas automatiquement sûr un MCP tiers.
- La publication stores, la signature et le pentest externe exigent des comptes/credentials/assessors externes.

Roadmap technique

- Corrélation source-to-data-flow plus profonde.
- Plus de fixtures d'évaluation et de reproduction offline.
- Workers cross-machine optionnels avec attestation et politiques distribuées.
- Beta réelle sur projets volumineux et pentest indépendant.

26. Liens et références

Ressource	Lien cliquable
Dépôt Hawk	https://github.com/MrBoodj011/Hawk-IDE
README	https://github.com/MrBoodj011/Hawk-IDE/blob/main/README.md
Architecture produit	https://github.com/MrBoodj011/Hawk-IDE/blob/main/PROJECT.md
Architecture technique	https://github.com/MrBoodj011/Hawk-IDE/blob/main/docs/architecture.md
Hawk AI natif	https://github.com/MrBoodj011/Hawk-IDE/blob/main/docs/native-ai.md
Smart MCP Brain	https://github.com/MrBoodj011/Hawk-IDE/blob/main/docs/smart-mcp.md
Docker orchestration	https://github.com/MrBoodj011/Hawk-IDE/blob/main/docs/parallel-orchestration.md
Sandbox reproduction	https://github.com/MrBoodj011/Hawk-IDE/blob/main/docs/sandbox-reproduction.md
Production readiness	https://github.com/MrBoodj011/Hawk-IDE/blob/main/docs/release/PRODUCTION_READINESS.md
Threat model	https://github.com/MrBoodj011/Hawk-IDE/blob/main/docs/security/THREAT_MODEL.md
External pentest runbook	https://github.com/MrBoodj011/Hawk-IDE/blob/main/docs/audit/EXTERNAL_PENTEST_RUNBOOK.md
Cybrense Hawk liaison	https://github.com/Cybrense-IT-Services/Hawk
MCP Specification	https://modelcontextprotocol.io/specification
MCP Apps	https://modelcontextprotocol.io/extensions/apps/overview
OWASP MCP Top 10	https://owasp.org/www-project-mcp-top-10/
A2A protocol	https://github.com/a2aproject/A2A
Ollama	https://github.com/ollama/ollama

Glossaire rapide

Terme	Définition
Code-OSS	Base open source de Visual Studio Code utilisée pour construire le desktop.
Worktree	Checkout Git isolé utilisé pour préparer un patch sans toucher le workspace réel.
MCP	Model Context Protocol: outils, ressources, prompts et apps structurés.
GoalSpec	Contrat typé de scope, autorité, budget, modèles et critères.
DAG	Graphe acyclique de tâches et dépendances.
ProofGraph	Graphe durable qui relie code, trafic, findings, preuves, patches et tests.
Signal	Détection qui nécessite encore validation et preuve.
Reproduction	Observation déterministe contrôlée; différente d'une vérification d'impact.
Gate	Condition obligatoire avant une transition de confiance.
BYOK	Bring Your Own Key: clé de provider conservée localement.
Loopback	Interface réseau locale 127.0.0.1/::1 non exposée au LAN.
SARIF	Format standard de résultats de code scanning.



27 / HANDOFF

27. Conclusion et point de départ

Conclusion

Hawk est déjà une plateforme locale cohérente: IDE brandé, agent AI review-controlled, AppSec passif, capture runtime, preuve, MCP gouverné et exécution Docker distribuée. Sa valeur vient surtout de la chaîne de contrôle entre le signal et la décision humaine, pas d'une promesse de scanner magique.

LOCAL données et control plane	REVIEW chaque patch visible	PROOF signal séparé du verdict	RECOVER sessions et workers durables
--	---------------------------------------	--	--

Par où commencer

- Ouvrir un workspace de test et lancer Index Security Surface.
- Configurer Ollama local ou un provider BYOK dans les réglages Hawk.
- Créer une petite tâche Hawk AI, inspecter le diff, lancer les gates puis décider Apply ou Reject.
- Importer un HAR de laboratoire ou paier le Browser Companion avec une regex étroite.
- Construire le worker Docker local puis tester une reproduction supportée.
- Copier la configuration MCP et créer une mission review-only avant d'essayer un workflow plus actif.

Projet: github.com/MrBoodj011/Hawk-IDE

Commande de validation complète: `npm run ci`